sol: a field specification from CSV rows to training batches

sol

SolData

`SolDataset.data` is a field-first dictionary: `data[field][g]` selects one field of one game. The integer g is a local game index; `data["gameid"][g]` is the original Oracle's Elixir game ID. All CSV cells remain **strings**, including numbers, flags, and empty cells (`""`). Numeric conversion, champion encoding, and tensors belong to the model configuration.

Access Forms — three value layouts

Scope	Access	Value	Semantics
Match	`data[f][g]`	`str`	One value from the first player row.
Player	`data[f][g][p]`	`str`	One of ten players, ordered by side and role.
Team	`data["team_"+f][g][s]`	`str`	Team-row value; s is an integer dictionary key.
Draft	`data[f][g][s]`	`str` or `list[str]`	Special fields below; no `team_` prefix.

Indices — role order is shared by all player fields

Side	top	jng	mid	bot	sup
Blue ($s=0$)	$p=0$	$p=1$	$p=2$	$p=3$	$p=4$
Red ($s=1$)	$p=5$	$p=6$	$p=7$	$p=8$	$p=9$

$p = 5s + r$, with $r \in \{0,1,2,3,4\}$ in the role order above. Team and draft values use `dict[int, ...]` with keys `0`, `1`; player values use `list[str]` of length ten.

Default Fields — present after a fresh extraction without extra requests

Field	Value at g	Semantics
`gameid`	`str`	Original game ID, distinct from local index g.
`date`	`str`	Timestamp, typically `YYYY-MM-DD HH:MM:SS`; used for splitting.
`patch`	`str`	Patch label, e.g. `"16.18"`; preserved as text.
`champion`	`list[str]`	Ten champion names in side/role order, not pick order.
`position`	`list[str]`	`["top", "jng", "mid", "bot", "sup"] * 2`.
`result`	`list[str]`	Ten player results: `"1"` for a win, `"0"` for a loss.
`team_result`	`dict[int, str]`	Team outcomes; `data["team_result"][g][0]` is Blue's label.
`firstPick`	`dict[int, str]`	Per-side first-pick flag, `"1"` or `"0"`. Empty cells default to Blue first.
`pick_order`	`dict[int, list[str]]`	Five names per side, copied from the team row's `pick1`–`pick5`.
`ban_order`	`dict[int, list[str]]`	Five names per side, copied from the team row's `ban1`–`ban5`.

`REQUIRED_FIELDS` names nine fields; `team_result` is also emitted because `result` is required. Pick and ban slots preserve each side's recorded order, not a merged sequence of draft turns.

CSV Field Catalog

The following catalog covers the shared 165-column header of the local 2020–2026 CSVs. Apart from the default fields, columns must be named in `requested_fields`. Spellings, spaces, capitalization, and parentheses are literal.

Match Fields — `data[f][g]`

Field	Semantics
`gameid`	Original game identifier.
`url`	Source match URL.
`league`	League or competition label.
`year`	Dataset year as text.
`split`	Split or season label.
`playoffs`	Playoff flag: `"1"` or `"0"`.
`date`	Game timestamp, copied from the first player row.
`game`	Game number within the series; raw cell, possibly empty.
`patch`	Patch label.
`datacompleteness`	`"complete"` or `"partial"`.

Player and Team Fields — request the CSV column name

For every CSV column f outside the ten match fields and `firstPick`, extraction creates `data[f][g]` as a ten-player list. It also creates `data["team_"+f][g]` from the two team rows, except for `champion` and `position`. Team cells are copied directly; they are not sums of player cells.

Field	Semantics
`participantid`	Source participant identifier; independent of the zero-based player index p.
`side`	`"Blue"` or `"Red"`.
`position`	Player role; no `team_position` is produced.
`playername`, `playerid`	Player display name and source ID. Team-row cells may be empty.
`teamname`, `teamid`	Player's team name and source ID. Their team-row keys are `team_teamname`, `team_teamid`.
`champion`	Champion name; no `team_champion` is produced.

Draft Columns — grouped fields are always available

Field	Semantics
`firstPick`	Special side dictionary, never a player list or `team_firstPick`. The CSV name is case-sensitive.
`pick1`, `pick2`, `pick3`,	Optional raw columns. Requesting one creates a player list and its `team_` counterpart. Player cells may be empty.
`pick4`, `pick5`	The default `pick_order` already groups the five team cells.
`ban1`, `ban2`, `ban3`,	Optional raw columns with the same player/team rule.
`ban4`, `ban5`	The default `ban_order` groups the five team cells.

Request Rules

```
data = SolDataset().extract_base_data(requested_fields=["kills", "teamname"]).data
g = next(iter(data["gameid"]))
blue_team = data["team_teamname"][g][0]
blue_kills = int(data["team_kills"][g][0])
```

Request `"kills"` to obtain both `kills` and `team_kills`; requesting only `"team_kills"` does not load it. Requests absent from the CSV header do not create populated fields. The nested `defaultdicts` create empty entries on missing lookups: check `f in data` and `g in data[f]` before reading uncertain keys.

Statistics

All fields on this and the next page use the player/team rule: `data[f][g][p]` and `data["team_"+f][g][s]`. Numbers remain strings. Available columns can contain empty cells, including on team rows and for metrics absent in older patches.

Result and Combat

Field	Semantics
`gamelength`, `result`	Duration in seconds; win/loss flag.
`kills`, `deaths`, `assists`	Recorded combat totals.
`teamkills`, `teamdeaths`	Team totals stored in these source columns; distinct from the `team_` prefix rule.
`doublekills`, `triplekills`, `quadrakills`, `pentakills`	Multi-kill counts.
`firstblood`	Team first-blood flag.
`firstbloodkill`, `firstbloodassist`, `firstbloodvictim`	Player first-blood involvement flags.
`team kpm`, `ckpm`	Team kills per minute; combined kills per minute. The space in `team kpm` is part of the name.

Objectives and Structures

Field	Semantics
`firstdragon`, `dragons`, `opp_dragons`	First-dragon flag; own and opposing dragon counts.
`elementaldrakes`, `opp_elementaldrakes`	Own and opposing elemental drake counts.
`infernals`, `mountains`, `clouds`, `oceans`, `chemtechs`, `hextechs`	Drake counts by element.
`dragons (type unknown)`	Dragon count without a known type; spaces and parentheses are part of the key.
`elders`, `opp_elders`	Own and opposing Elder Dragon counts.
`firstherald`, `heralds`, `opp_heralds`	First-Herald flag; own and opposing Herald counts.
`void_grubs`, `opp_void_grubs`	Own and opposing Void Grub counts.
`firstbaron`, `barons`, `opp_barons`	First-Baron flag; own and opposing Baron counts.
`atakhans`, `opp_atakhans`	Own and opposing Atakhan counts.
`firsttower`, `towers`, `opp_towers`	First-tower flag; own and opposing tower counts.
`firstmidtower`, `firsttothreetowers`	First mid tower; first team to three towers flags.
`turretplates`, `opp_turretplates`	Own and opposing turret plate counts.
`inhibitors`, `opp_inhibitors`	Own and opposing inhibitor counts.

Damage and Vision

Field	Semantics
`damagetochampions`, `dpm`, `damageshare`	Champion damage, damage per minute, and damage share.
`damagetakenperminute`, `damagemitigatedperminute`	Damage taken and mitigated per minute.
`damagetotowers`	Damage to towers.
`wardsplaced`, `wpm`, `wardskilled`, `wcpm`	Wards placed and cleared, with per-minute rates.
`controlwardsbought`	Control wards purchased.
`visionscore`, `vspm`	Vision score and vision score per minute.

Gold and Farming

Field	Semantics
`totalgold`, `earnedgold`, `goldspent`	Total, earned, and spent gold.
`earned gpm`, `earnedgoldshare`	Earned gold per minute; share of team earned gold.
`gspd`, `gpr`	Provider's gold advantage metrics, copied unchanged.
`total cs`, `cspm`	Total creep score and creep score per minute. The space in `total cs` is literal.
`minionkills`, `monsterkills`	Lane minion and neutral monster kills.
`monsterkillsownjungle`, `monsterkillsenemyjungle`	Monster kills in the own and opposing jungle.

Timed Statistics — literal names at 10, 15, 20, and 25 minutes

Quantity	10 min	15 min	20 min	25 min
Gold	`goldat10`	`goldat15`	`goldat20`	`goldat25`
Experience	`xpat10`	`xpat15`	`xpat20`	`xpat25`
Creep score	`csat10`	`csat15`	`csat20`	`csat25`
Opponent gold	`opp_goldat10`	`opp_goldat15`	`opp_goldat20`	`opp_goldat25`
Opponent XP	`opp_xpat10`	`opp_xpat15`	`opp_xpat20`	`opp_xpat25`
Opponent CS	`opp_csat10`	`opp_csat15`	`opp_csat20`	`opp_csat25`
Gold difference	`golddiffat10`	`golddiffat15`	`golddiffat20`	`golddiffat25`
XP difference	`xpdiffat10`	`xpdiffat15`	`xpdiffat20`	`xpdiffat25`
CS difference	`csdiffat10`	`csdiffat15`	`csdiffat20`	`csdiffat25`
Kills	`killsat10`	`killsat15`	`killsat20`	`killsat25`
Assists	`assistsat10`	`assistsat15`	`assistsat20`	`assistsat25`
Deaths	`deathsat10`	`deathsat15`	`deathsat20`	`deathsat25`
Opponent kills	`opp_killsat10`	`opp_killsat15`	`opp_killsat20`	`opp_killsat25`
Opponent assists	`opp_assistsat10`	`opp_assistsat15`	`opp_assistsat20`	`opp_assistsat25`
Opponent deaths	`opp_deathsat10`	`opp_deathsat15`	`opp_deathsat20`	`opp_deathsat25`

`opp_` denotes the opposing player/team as recorded by the provider. Difference fields are recorded own-minus-opponent values. Timed cells can be empty when the game ends before the timestamp or the source lacks detailed statistics. Do not infer numeric zero from an empty cell.

Team Naming — the prefix is always prepended verbatim

Request	Player Access	Team Access
`"teamname"`	`data["teamname"][g][p]`	`data["team_teamname"][g][s]`
`"teamkills"`	`data["teamkills"][g][p]`	`data["team_teamkills"][g][s]`
`"team kpm"`	`data["team kpm"][g][p]`	`data["team_team kpm"][g][s]`
`"earned gpm"`	`data["earned gpm"][g][p]`	`data["team_earned gpm"][g][s]`
`"total cs"`	`data["total cs"][g][p]`	`data["team_total cs"][g][s]`